

How to Design Classes

A step-by-step way to plan the architecture before writing code

That's normal. Writing code is a skill you already have. Design is a separate skill, and it gets easier when you follow the same steps every time.

A 6-step process

1. Describe the system in plain English first, with no code.

Write 3–5 sentences:

“Kelly is a person. She gets hungry and tired over time. When she eats, her hunger goes down. When she sleeps, her energy comes back. We want to check her status.”

2. Circle the nouns. They're your candidate classes.

Kelly/Person, hunger, energy, food, status.

Not every noun becomes a class. Ask: *does it have its own data AND its own behavior?*

- Hunger has a level and rules (eat, check if hungry), so it's a class (Stomach).
- Status is just a printout, so it's a method, not a class.
- Food? It could be a class if foods differ (calories, type). Otherwise it's just a number.

3. Underline the verbs. They're your methods.

eats, sleeps, gets hungry, check status.

Then ask *who owns this verb?* Put the method on the class whose data it changes:

- `eat ( )` changes the hunger level, so it goes on Stomach.
- `sleep ( )` changes energy, so it goes on Energy.
- Person gets thin methods that hand off to them (`person . feed ( )` calls `stomach . eat ( )`).

4. Connect the classes using relationship sentences.

Say each pair out loud:

Sentence that sounds right	Relationship	Code
“A is a B”	Inheritance	<code>class Dog(Animal)</code>
“A has a B, and B dies with A”	Composition	A creates B in <code>__init__</code>
“A has a B, but B lives on its own”	Aggregation	B is passed into A
“A uses B briefly”	Dependency	B is a method parameter

If none of these sentences sounds natural, the two classes probably shouldn't be connected. That's how the `Kelly_feelings` has a `Kelly` case gives itself away.

5. Check each class with one question: “What is this class responsible for?”

If the answer has an “and” in it, the class may be doing too much. Split it. This is the Single Responsibility Principle, and it's the most useful rule in LLD.

6. *Sketch it before you code it.*

Boxes and arrows on paper, or just a list:

```
Person
- name
- stomach: Stomach    (composition)
- energy: Energy      (composition)
+ feed(), sleep(), status()
```

```
Stomach
- hunger_lvl
+ eat(), is_hungry()
```

Only then write Python. Changing the design on paper takes seconds. Changing it in code is slow.

Habits that help

- **Start too simple, then split.** Begin with one class. When it gets messy or does two jobs, pull a piece out. Don't try to design the perfect structure upfront.
- **Prefer composition over inheritance.** Only use inheritance when “is-a” is clearly true *and* the child really is a specialized version of the parent.
- **Data and the code that changes it live together.** If class A keeps reaching into `b.something.level` to make decisions, that logic probably belongs inside B.
- **Name things after real-world concepts.** Stomach is clearer than `KellyHungerManager`. If you can't find a good name, the class may not be a real concept.

Practice

Take a small system, such as a parking lot, a library, a vending machine or a coffee shop, and run steps 1–6 before writing any code. These are the classic LLD interview problems for exactly this reason. Do 5–10 of them and the process becomes automatic.